



TECHNICAL UNIVERSITY OF DENMARK

Exercise 3 - Mutual Information-based Registration

Date of submission: 7th of October, 2025

Submitted By:

Holger Max Fløe Lyng, S214776

Maja Reben Johannesen, S204287

Dorthea Nørregaard, S204295

Othilia Wagner, S214780

Philip Vestergaard-Laustsen, s196248

Table of Contents

1	Introduction	2
2	Task 1 - Resample the T2-weighted scan to the image grid of the T1-weighted scan	3
2.1	Implementation of Task 1	4
3	Task 2 - Compute and visualize the joint histogram	5
3.1	Implementation of Task 2	7
4	Task 3 - Compute the mutual information between the two images	8
4.1	Implementation of Task 3	9
5	Task 4 - Evaluate the mutual information across a range of rotation angles	10
5.1	Implementation of Task 4	12
6	Task 5 - Perform automatic registration	14
6.1	Implementation of Task 5	16
7	Overview of all results	19
8	Task 6 - Compare the joint histogram before and after registration	20
8.1	Implementation of Task 6	21
9	Additional task - Compare the MI obtained with landmark-based and intensity-based registration	22
9.1	Implementation of additional task	25
10	Conclusion	27

1 Introduction

In this exercise, we utilize the mutual information measurement to find the best rigid registration.

This is a very effective way to do registration as we don't need to manually annotate the images and we thereby heighten the accuracy and effectivity.

Intensity-based registration is a way to automatically do the registrations by minimizing energies that are calculated directly from the intensities of the raw images.

$$E(\mathbf{w}) = \sum_{n=1}^N [\mathcal{F}(\mathbf{x}_n) - \mathcal{M}(y(\mathbf{x}_n, \mathbf{w}))]^2. \quad (1)$$

Here, $\mathbf{x}_n$ represents the world coordinates of voxels in the fixed image, and $\mathcal{F}(\mathbf{x}_n)$ denotes their corresponding image intensities. Likewise, $\mathcal{M}(y(\mathbf{x}_n, \mathbf{w}))$ represents the intensities in the moving image, evaluated at the transformed locations $y(\mathbf{x}_n, \mathbf{w})$. Often the transformed positions don't coincide with exact voxel center in the moving image and therefore interpolation is needed. Also if a mapped location lies outside of the spatial view of the moving image, a constant intensity will usually be assigned (often 0).

If we have two images that come from two different imaging modalities, the intensity profiles between the images can be quite different. When having this we can still do an automatic registration by optimizing the mutual information (MI) between the two images.

MI measures the amount of information that one image provides about another. In the context of image registration, MI quantifies the statistical dependence between the intensities of the fixed image $\mathcal{F}$ and the moving image $\mathcal{M}$ after applying a spatial transformation parameterized by $\mathbf{w}$.

In this exercise we use a T1 and T2 MRI image. First we map the T2 into the image space of the T1 and then we will use mutual based information to register them to each other.

2 Task 1 - Resample the T2-weighted scan to the image grid of the T1-weighted scan

In this task you perform the resampling just like in the landmark-based registration exercise.

Find the voxel-to-voxel transformation between $\mathbf{v}_{T1}$ and $\mathbf{v}_{T2}$:

$$\begin{pmatrix} \mathbf{v}_{T2} \\ 1 \end{pmatrix} = \mathbf{M}_{T2}^{-1} \cdot \mathbf{M}_{T1} \cdot \begin{pmatrix} \mathbf{v}_{T1} \\ 1 \end{pmatrix}$$

At the location $\mathbf{v}_{T2}$, you should then use cubic B-spline interpolation to determine the intensity in the T2-weighted scan, and store it at index $\mathbf{v}_{T1}$ in the newly created image.

In this task we use what we learned in exercise 2. We get the T2 image into the T1 space by interpolation.

Figure 1 shows the T2 image interpolated to T1. From the figure it is seen that the T2 image is rotated around the Z-axis.

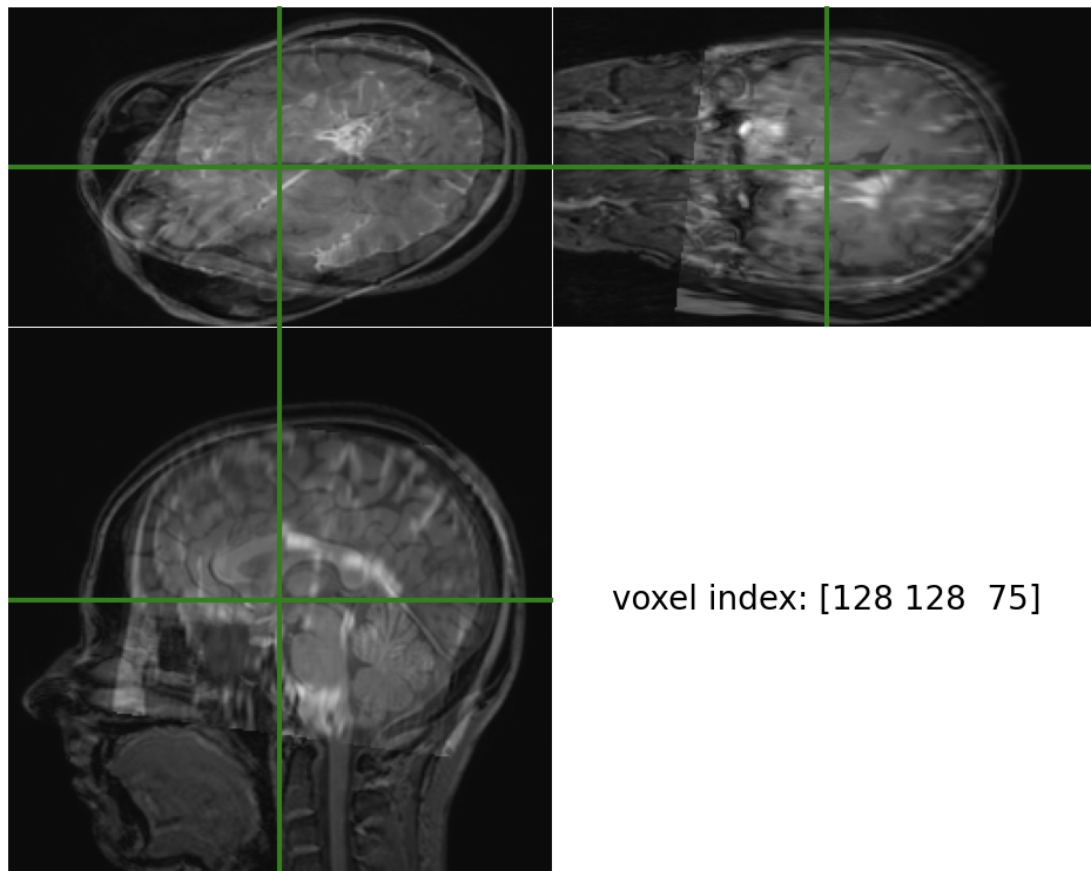


Figure 1: T2 interpolated to T1 space

2.1 Implementation of Task 1

The code is a copy from last weeks exercise:

```
1 #Finding the affine matrices
2 A_T1 = T1.affine
3 A_T2 = T2.affine
4
5 #Creating the V_stack to
6 V1,V2,V3 = np.meshgrid( np.arange( T1_data.shape[0] ), np.arange(
   ↳ T1_data.shape[1] ), np.arange( T1_data.shape[2] ), indexing='ij' )
7 V_stack = []
8 V_stack.append( V1.flatten() )
9 V_stack.append( V2.flatten() )
10 V_stack.append( V3.flatten() )
```

```
11 V_stack.append( np.ones( V1.size ) )
12 V_stack = np.array(V_stack)
13
14 V_T2_space = np.linalg.inv( A_T2 ) @ A_T1 @ V_stack
15
16 #Now we interpolate the T2 data to the T1 grid
17 V_T2_in_T1_space_intensity = scipy.ndimage.map_coordinates( T2_data,
    ↪ V_T2_space[:3,:] )
18 V_T2_in_T1_space_intensity = V_T2_in_T1_space_intensity.reshape(
    ↪ T1_data.shape )
19 T2_data_resampled = V_T2_in_T1_space_intensity
```

3 Task 2 - Compute and visualize the joint histogram

Compute the joint histogram

$$\mathbf{H} = \begin{pmatrix} h_{1,1} & \dots & h_{1,B} \\ \vdots & \dots & \vdots \\ h_{B,1} & \dots & h_{B,B} \end{pmatrix}$$

of the T1-weighted scan and the resampled T2-weighted scan you just created, using $B = 32$ bins. In order not to let the registration process be dominated by the (extremely numerous) background voxels, ignore all voxels with an intensity lower than 10 in either image when computing the histogram. Visualize the joint histogram using a 3D bar plot, and label the axes.

To estimate the statistical dependence between the intensities of the two images, we construct a *joint histogram* from corresponding voxel pairs in the two images, given by

$$(\mathcal{F}(\mathbf{x}_n), \mathcal{M}(y(\mathbf{x}_n, \mathbf{w}))),$$

Each voxel pair contributes to a bin in the joint histogram H , which counts how often intensity f in the fixed image coincides with intensity m in the moving image.

From this histogram, we can estimate the joint probability distribution $p(f, m)$ and the marginal

entropies $p(f)$ and $p(m)$ by summing over one axis.

$$E(\mathbf{w}) = H_{F,M} \quad \text{with} \quad H_{F,M} = - \sum_{f=1}^B \sum_{m=1}^B p_{f,m} \log(p_{f,m}) \quad (2)$$

$H_{F,M}$ is also known as the joint entropy. In the context of image registration, joint entropy becomes larger when the joint histogram is “smeared out,” which happens when the images are poorly aligned. This means that $H_{F,M}$ can serve as an energy function to guide the automatic registration process toward better alignment.

Figure 2 shows the joint histogram between the T1 image and the resampled T2 image.

Our joint histogram is normalised, which makes it easier to compare across images. We can observe a high number of overlapping background (a high joint probability of intensity 0). If we had a good alignment our intensity combination would have small counts and be concentrated around a few bins in the histogram. As we can see in figure 2 this is not the case, and it indicates that a better alignment can be found.

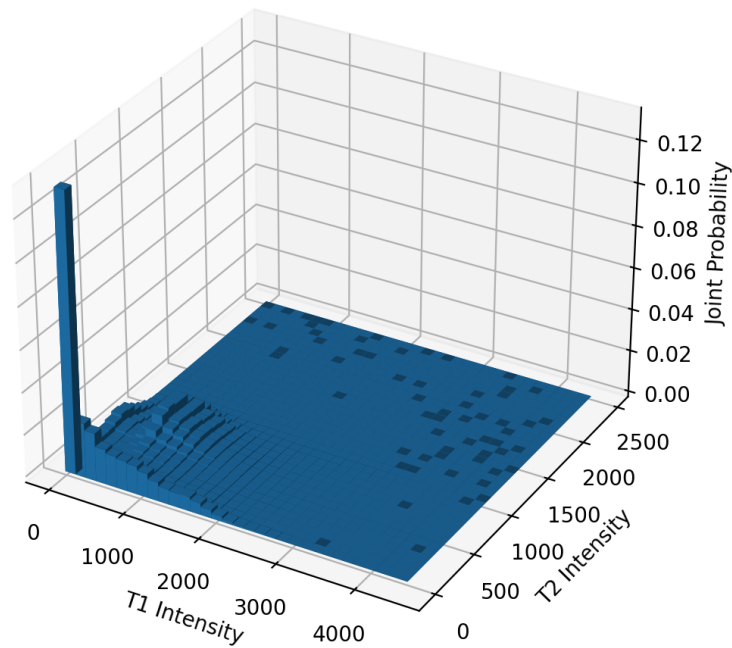


Figure 2: Joint histogram between the T1 and the resampled T2 image.

3.1 Implementation of Task 2

```
1  #Initializing the bin number and threshold
2  B = 32
3  threshold = 10
4
5  #Finding the joint histogram
6  H, xedges, yedges = np.histogram2d( T1_data.flatten(),
7  T2_data_resampled.flatten(), bins=B, range=[[threshold, np.max(T1_data)],
8  [threshold, np.max(T2_data_resampled)]] )
9  jointProb = H / np.sum(H)
10
11 #Taken inspiration from:
12 #https://stackoverflow.com/questions/52385299/plot-a-3d-bar-histogram
13 xpos, ypos = np.meshgrid(xedges[:-1]+xedges[1:], yedges[:-1]+yedges[1:])
14     - (xedges[1]-xedges[0])
15
```

```

16
17 xpos = xpos.flatten()*1./2 #take times 1/2 to get to the center of the
   ↪ bin
18 ypos = ypos.flatten()*1./2
19 xpos = xpos.ravel()
20 ypos = ypos.ravel()
21 zpos = np.zeros_like(xpos)
22
23
24 dx = xedges [1] - xedges [0]
25 dy = yedges [1] - yedges [0]
26 dz = jointProb.ravel()
27
28 fig, ax = plt.subplots(subplot_kw=dict(projection='3d'), figsize=(8,6))
29 ax.bar3d(xpos, ypos, zpos, dx, dy, dz, shade=True)
30
31 ax.set_xlabel('T1 Intensity')
32 ax.set_ylabel('T2 Intensity')
33 ax.set_zlabel('Joint Probability')
34 plt.show()

```

4 Task 3 - Compute the mutual information between the two images

Write a function that takes two image volumes defined on the same image grid as input, and returns the mutual information between the two images:

$$MI = H_F + H_M - H_{F,M}$$

with

$$H_{F,M} = - \sum_{f=1}^B \sum_{m=1}^B p_{f,m} \log(p_{f,m}),$$

$$H_F = - \sum_{f=1}^B p_f \log(p_f),$$

and

$$H_M = - \sum_{m=1}^B p_m \log(p_m).$$

Your function should make use of the joint histogram computed as in the previous task (i.e., using $B = 32$ bins and ignoring all voxels with intensity lower than 10).

Use your new function to compute the mutual information between the T1-weighted image and the resampled T2-weighted image.

To create the function we use the equations given in the task description.

Before doing any registration/rotation our mutual information function calculated the MI between the T1 image and the resampled T2 image to be approximately -0.15.

4.1 Implementation of Task 3

```
1  #Creation of the function calculating the mutual information between two  
2  ↪ images  
3  def mutual_information(B, T1_data, T2_data_resampled, threshold):  
4      #Finding the joint histogram  
5      H, x, y = np.histogram2d( T1_data.flatten(),  
6      ↪ T2_data_resampled.flatten(), bins=B, range=[[threshold,  
7      ↪ np.max(T1_data)], [threshold, np.max(T2_data_resampled)]] )  
8      jointProb = H / np.sum(H)  
9      H_FM = -np.sum( jointProb * np.log( jointProb + 1e-12 ) ) #adding a  
10     ↪ small number to avoid log(0)  
11  
12     #finding the histogram for T1  
13     H_T1, x = np.histogram( T1_data.flatten(), bins=B, range=(threshold,  
14     ↪ np.max(T1_data)) )  
15     p_F = H_T1 / np.sum(H_T1)  
16     H_F = -np.sum( p_F * np.log( p_F + 1e-12 ) ) #adding a small number  
17     ↪ to avoid log(0)  
18  
19     #finding the histogram for T2  
20     H_T2, x = np.histogram( T2_data_resampled.flatten(), bins=B,  
21     ↪ range=(threshold, np.max(T2_data_resampled)) )  
22     p_M = H_T2 / np.sum(H_T2)  
23     H_M = -np.sum( p_M * np.log( p_M + 1e-12 ) ) #adding a small number  
24     ↪ to avoid log(0)  
25  
26     #Calculating the mutual information  
27     MI= H_F + H_M - H_FM
```

```
return MI
```

5 Task 4 - Evaluate the mutual information across a range of rotation angles

Implement a grid search over a range of rotation angles, with the goal of identifying the angle with approximately the highest mutual information (i.e., where the registration is best). Make an educated guess of a suitable range by visually inspecting the images (or by simply trying a few rotation angles and inspecting the result), and then define a list of candidate angles at intervals of e.g., 5° apart:

```
a = np.arange(a_lowest, a_highest, 5)
```

Loop over all candidate rotation angles, each time (1) creating a corresponding rotation matrix $\mathbf{R}$; (2) resampling the T2-weighted volume accordingly; (3) calculating the mutual information with the function you created above; and (4) storing the mutual information value.

During task 1 and by trying the different rotation matrix it was found that we needed to rotate the moving image around the z-axis. This means that our rotation matrix is

$$\mathbf{R} = \mathbf{R}_z = \begin{pmatrix} \cos(\theta_z) & -\sin(\theta_z) & 0 \\ \sin(\theta_z) & \cos(\theta_z) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Figure 3 shows the variation of MI as a function of the rotation angle. The MI curve shows a clear maximum at 45 degrees, with a calculated $MI = 0.0142$. The result corresponds well with the known simulated rotation of the T2 volume around the z-axis. The angle indicates the highest statistical dependency between the two images, and therefore also the best registration.

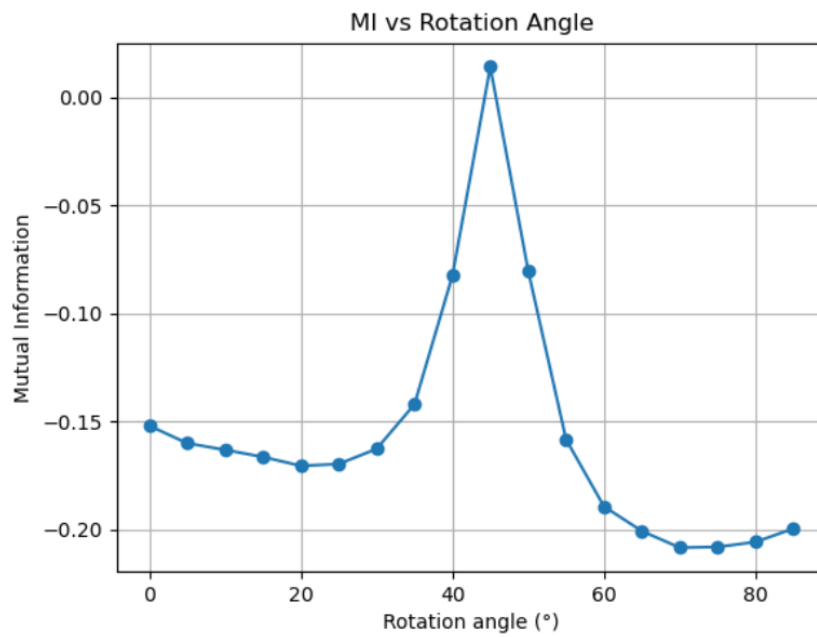


Figure 3: Mutual Information as a function of rotation angle.

Figure 4 illustrates the overlay between the resampled T2 image and the T1 image at the optimal rotation of 45 degrees. The anatomical structures align well, particularly the cortical boundaries and the ventricles, indicating a successful registration.



Figure 4: Overlay of the T1-weighted image and the resampled T2-weighted image at the optimal rotation angle 45 degrees.

5.1 Implementation of Task 4

```
1 a = np.arange(0,90, 5)
2 mi_values = []
3 resampled_images = []
4
5
6 for i in range(len(a)):
7     #Creating the rotation matrix R
8     R_z = np.array( [ [ np.cos(np.deg2rad(a[i])),
9         ↪ -np.sin(np.deg2rad(a[i])), 0 ],
10        [ np.sin(np.deg2rad(a[i])),
11        ↪ np.cos(np.deg2rad(a[i])), 0 ],
12        [ 0, 0, 1 ] ] )
13
14     R_y = np.array( [ [ np.cos(np.deg2rad(a[i])),0,
15         ↪ -np.sin(np.deg2rad(a[i])), ],
```

5 TASK 4 - EVALUATE THE MUTUAL INFORMATION ACROSS A RANGE OF ROTATION ANGLES

```
13         [ 0, 1, 0 ],
14         [ np.sin(np.deg2rad(a[i])),0,
           ↪ np.cos(np.deg2rad(a[i])) ] ] )
15 R_x = np.array( [ [ 1, 0, 0 ],
16                 [ 0, np.cos(np.deg2rad(a[i])),
           ↪ -np.sin(np.deg2rad(a[i])) ],
17                 [ 0, np.sin(np.deg2rad(a[i])),
           ↪ np.cos(np.deg2rad(a[i])) ] ] )
18
19 #By inspecting found out I only need to rotate around z
20 R = R_z #R_z @ R_y @ R_x
21
22 #Resample the T2 data
23 #Creating the M matrix from the book where it says A=R
24 M = np.eye(4)
25 M[:3,:3] = R
26
27 #then we do like last time
28 #Creating the V_stack to
29 V1,V2,V3 = np.meshgrid( np.arange( T1_data.shape[0] ), np.arange(
           ↪ T1_data.shape[1] ), np.arange( T1_data.shape[2] ), indexing='ij'
           ↪ )
30 V_stack = []
31 V_stack.append( V1.flatten() )
32 V_stack.append( V2.flatten() )
33 V_stack.append( V3.flatten() )
34 V_stack.append( np.ones( V1.size ) )
35 V_stack = np.array(V_stack)
36
37 #do the rotation
38 V_T2_space = np.linalg.inv( A_T2 ) @ M @ A_T1 @ V_stack #With the M
           ↪ for rotation
39
40 #Now we interpolate the T2 data to the T1 grid
41 V_T2_in_T1_space_intensity = scipy.ndimage.map_coordinates( T2_data,
           ↪ V_T2_space[:3,:] )
42 V_T2_in_T1_space_intensity = V_T2_in_T1_space_intensity.reshape(
           ↪ T1_data.shape )
43 T2_data_resampled_a = V_T2_in_T1_space_intensity
44 resampled_images.append(T2_data_resampled_a)
45
46 #Calculating the MI
47 mi = mutual_information(32, T1_data, T2_data_resampled_a, 10)
48 mi_values.append(mi)
49
```

```

50     #Viewer( T2_data_resampled_a / T2_data_resampled_a.max() + T1_data /
    ↪     T1_data.max() )
51
52     #Finding the best angle
53     best_idx = np.argmax(mi_values)
54     best_angle = a[best_idx]
55     print(f"\nBest rotation angle: {best_angle}° with MI =
    ↪     {mi_values[best_idx]:.4f}")
56
57     # --- Plot MI vs angles ---
58     plt.figure()
59     plt.plot(a, mi_values, '-o')
60     plt.xlabel("Rotation angle (°)")
61     plt.ylabel("Mutual Information")
62     plt.title("MI vs Rotation Angle")
63     plt.grid(True)
64     plt.show()
65
66     # --- Plot overlay for best angle ---
67     best_resampled = resampled_images[best_idx]
68     z = T1_data.shape[2] // 2 # Middle slice
69     overlay_best = (best_resampled[:, :, z] / best_resampled.max()) +
    ↪     (T1_data[:, :, z] / T1_data.max())
70
71     plt.figure(figsize=(8, 8))
72     plt.imshow(overlay_best.T, cmap='gray', origin='lower')
73     plt.title(f'Overlay of T1 and Resampled T2 (middle slice) -- Best angle:
    ↪     {best_angle}°')
74     plt.axis('off')
75     plt.show()
76
77

```

6 Task 5 - Perform automatic registration

Plot the negative mutual information, i.e. the energy function $E(\mathbf{w}) = H_{F,M} - H_F - H_M$ for every angle in your grid search space. Select the one with the lowest energy (i.e., the best angle for registration) and transform the T2-weighted volume according to this angle. Use the Viewer() function to visualize your registration result.

$H_F + H_M - H_{F,M}$ is called mutual information — a measure of how much two images share information. For image registration, we use the negative of this measure as an energy function. So when minimizing this it helps to align the images.

$$E(\mathbf{w}) = H_{F,M} - H_F - H_M$$

To do this in practice, we use numerical optimization methods that adjust the alignment parameters $\mathbf{w}$ to make the energy as low as possible. So in other words we want to find the most negative $E(\mathbf{w})$.

As shown in Figure 5, the curve of $E(\mathbf{w})$ versus the rotation angle is the mirrored image of the MI curve in Figure 3. It has a clear minimum also at 45 degrees and a calculated $MI = -0.0142$. This confirms that the MI maximum and the $E(\mathbf{w})$ coincide, as it was theoretically expected.

The overlay in Figure 6 displays the automatically registered T2 volume at the optimal angle (45 degrees). The anatomical match between T1 and T2 is alike the one from Task 4, which demonstrates that the automatic registration using $E(\mathbf{w})$ identifies the same transformation as the direct MI optimization.

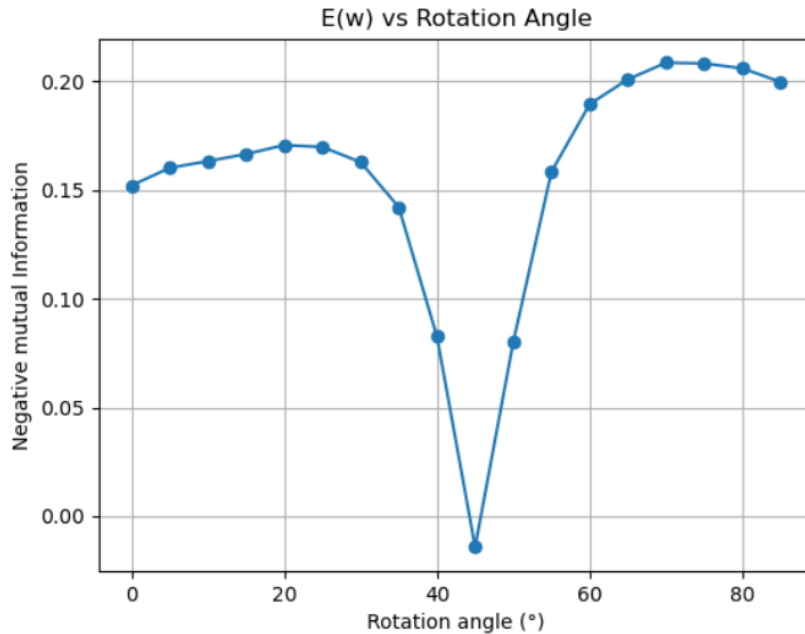


Figure 5: Negative mutual information $E(\mathbf{w})$ as a function of rotation angle.

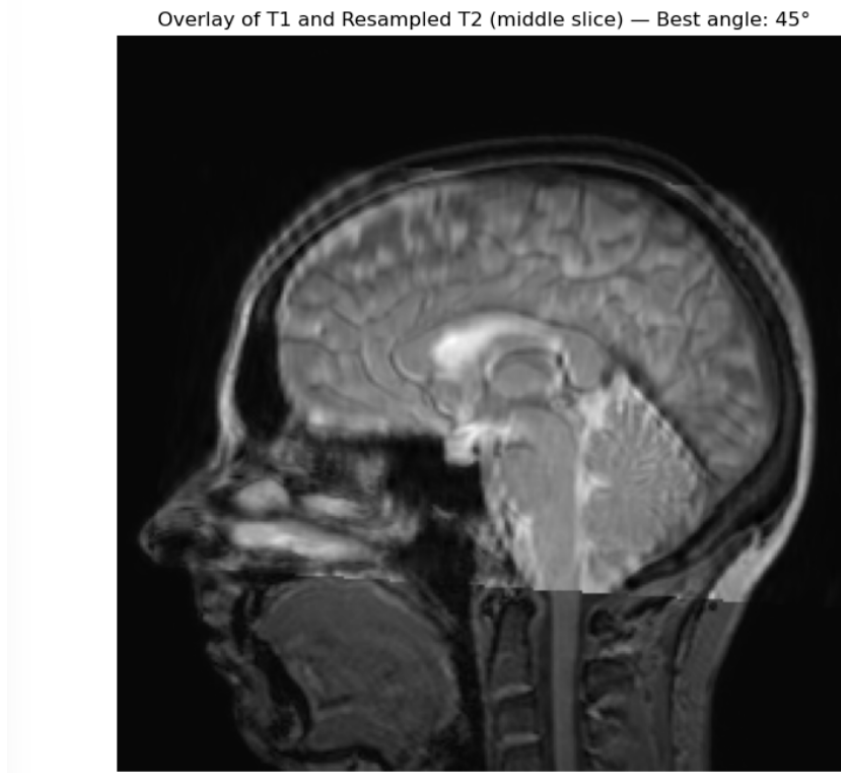


Figure 6: Overlay of the T1-weighted image and the automatically registered T2-weighted image at the optimal rotation angle(45 degrees).

6.1 Implementation of Task 5

We start by creating a function that calculates the negative mutual information.

```
1 #Creating the energy function
2 def energy(B, T1_data, T2_data_resampled, threshold):
3     #Finding the joint histogram
4     H, x, y = np.histogram2d( T1_data.flatten(),
5                               ↪ T2_data_resampled.flatten(), bins=B, range=[[threshold,
6                               ↪ np.max(T1_data)], [threshold, np.max(T2_data_resampled)]] )
7     jointProb = H / np.sum(H)
8     H_FM = -np.sum( jointProb * np.log( jointProb + 1e-12 ) ) #adding a
9     ↪ small number to avoid log(0)
10
11     #finding the histogram for T1
12     H_T1, x = np.histogram( T1_data.flatten(), bins=B, range=(threshold,
13     ↪ np.max(T1_data)) )
14     p_F = H_T1 / np.sum(H_T1)
15     H_F = -np.sum( p_F * np.log( p_F + 1e-12 ) ) #adding a small number
16     ↪ to avoid log(0)
```

```

12
13     #finding the histogram for T2
14     H_T2, x = np.histogram( T2_data_resampled.flatten(), bins=B,
15         ↪ range=(threshold, np.max(T2_data_resampled)) )
16     p_M = H_T2 / np.sum(H_T2)
17     H_M = -np.sum( p_M * np.log( p_M + 1e-12 ) ) #adding a small number
18         ↪ to avoid log(0)
19
20     #Calculating the mutual information
21     E_w= H_FM - H_F - H_M
22
23     return E_w

```

We can now do as in task 4 and find at which angle we have the lowest negative mutual information.

```

1  a = np.arange(0,90, 5)
2  ew_values = []
3  resampled_images = []
4
5
6  for i in range(len(a)):
7      #Creating the rotation matrix R
8      R_z = np.array( [ [ np.cos(np.deg2rad(a[i])),
9         ↪ -np.sin(np.deg2rad(a[i])), 0 ],
10         [ np.sin(np.deg2rad(a[i])),
11         ↪ np.cos(np.deg2rad(a[i])), 0 ],
12         [ 0, 0, 1 ] ] )
13
14      R_y = np.array( [ [ np.cos(np.deg2rad(a[i])),0,
15         ↪ -np.sin(np.deg2rad(a[i])), ],
16         [ 0, 1, 0 ],
17         [ np.sin(np.deg2rad(a[i])),0,
18         ↪ np.cos(np.deg2rad(a[i])) ] ] )
19
20      R_x = np.array( [ [ 1, 0, 0 ],
21         [ 0, np.cos(np.deg2rad(a[i])),
22         ↪ -np.sin(np.deg2rad(a[i])) ],
23         [ 0, np.sin(np.deg2rad(a[i])),
24         ↪ np.cos(np.deg2rad(a[i])) ] ] )
25
26      #By inspecting found out I only need to rotate around z
27      R = R_z @ R_y @ R_x
28
29      #Resample the T2 data

```

```
23  #Creating the M matrix from the book where it says A=R
24  M = np.eye(4)
25  M[:3,:3] = R
26
27  #then we do like last time
28  #Creating the V_stack to
29  V1,V2,V3 = np.meshgrid( np.arange( T1_data.shape[0] ), np.arange(
    ↪ T1_data.shape[1] ), np.arange( T1_data.shape[2] ), indexing='ij'
    ↪ )
30  V_stack = []
31  V_stack.append( V1.flatten() )
32  V_stack.append( V2.flatten() )
33  V_stack.append( V3.flatten() )
34  V_stack.append( np.ones( V1.size ) )
35  V_stack = np.array(V_stack)
36
37  #do the rotation
38  V_T2_space = np.linalg.inv( A_T2 ) @ M @ A_T1 @ V_stack #With the M
    ↪ for rotation
39
40  #Now we interpolate the T2 data to the T1 grid
41  V_T2_in_T1_space_intensity = scipy.ndimage.map_coordinates( T2_data,
    ↪ V_T2_space[:3,:] )
42  V_T2_in_T1_space_intensity = V_T2_in_T1_space_intensity.reshape(
    ↪ T1_data.shape )
43  T2_data_resampled_a = V_T2_in_T1_space_intensity
44  resampled_images.append(T2_data_resampled_a)
45
46  #Calculating the MI
47  ew = energy(32, T1_data, T2_data_resampled_a, 10)
48  ew_values.append(ew)
49
50  #Viewer( T2_data_resampled_a / T2_data_resampled_a.max() + T1_data /
    ↪ T1_data.max() )
51
52
53
54  # Find best angle
55  best_idx = np.argmin(ew_values)
56  best_angle = a[best_idx]
57  print(f"\nBest rotation angle: {best_angle}° with MI =
    ↪ {ew_values[best_idx]:.4f}")
58
59  # --- Plot MI vs angles ---
60  plt.figure()
```

```
61 plt.plot(a, ew_values, '-o')
62 plt.xlabel("Rotation angle (°)")
63 plt.ylabel("Negative mutual Information")
64 plt.title("E(w) vs Rotation Angle")
65 plt.grid(True)
66 plt.show()
67
68 # --- Plot overlay for best angle ---
69 best_resampled_ew = resampled_images[best_idx]
70 z = T1_data.shape[2] // 2 # Middle slice
71 overlay_best_ew = (best_resampled_ew[:, :, z] / best_resampled_ew.max())
72 ↪ + (T1_data[:, :, z] / T1_data.max())
73
74 plt.figure(figsize=(8, 8))
75 plt.imshow(overlay_best_ew.T, cmap='gray', origin='lower')
76 plt.title(f'Overlay of T1 and Resampled T2 (middle slice) -- Best angle:
77 ↪ {best_angle}°')
78 plt.axis('off')
79 plt.show()
```

7 Overview of all results

To see how the registrations are between the different methods an overview has been made.

As it can be observed in figure 7 when comparing to the T1 image the registration has been successful when using Mutual Information and its negative.

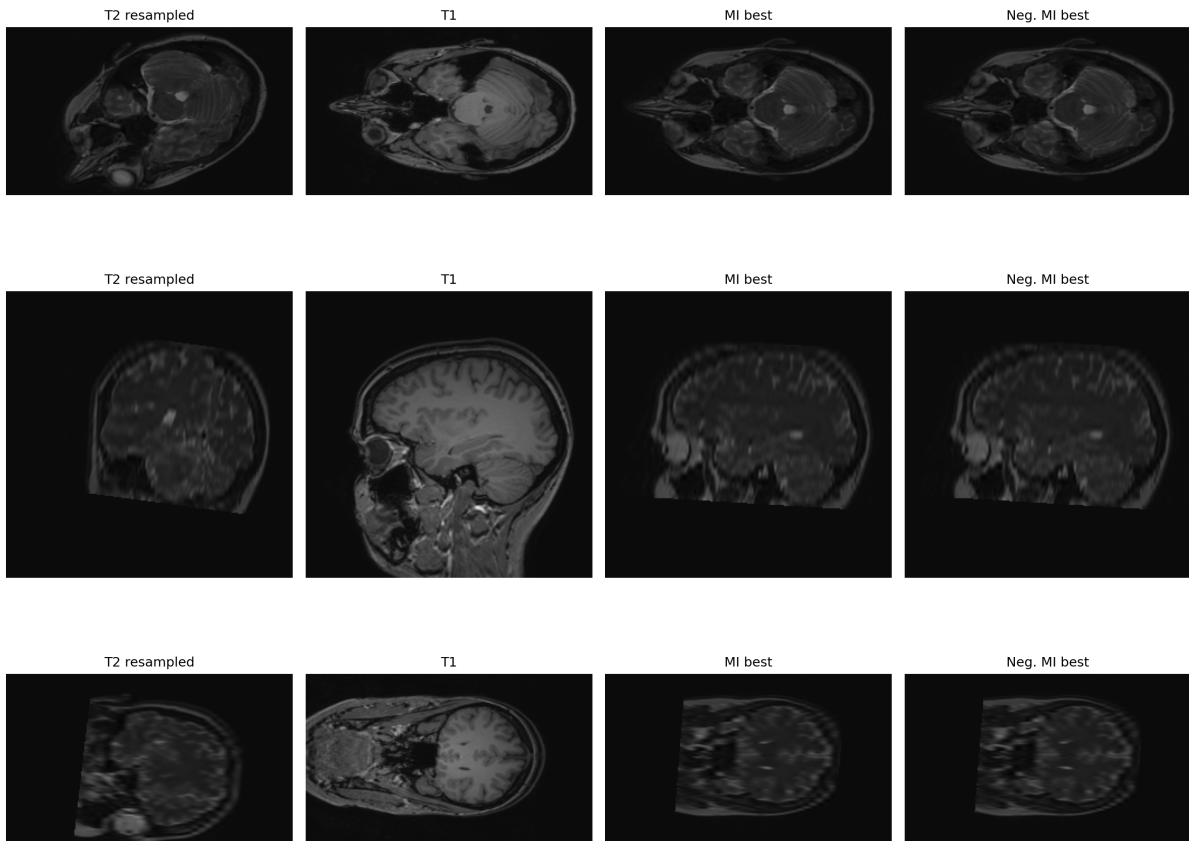


Figure 7: Overview of the different registered T2 images when using the different methods.

8 Task 6 - Compare the joint histogram before and after registration

Visualize the joint histogram after registration. Compare the result to your initial joint histogram (i.e., before registration) and discuss your findings in your report.

To further evaluate the quality of the registration, we computed joint histograms for the original resampled T2 image, the MI-based registration result, and the negative-MI registration. The

histograms are seen in Figure 8

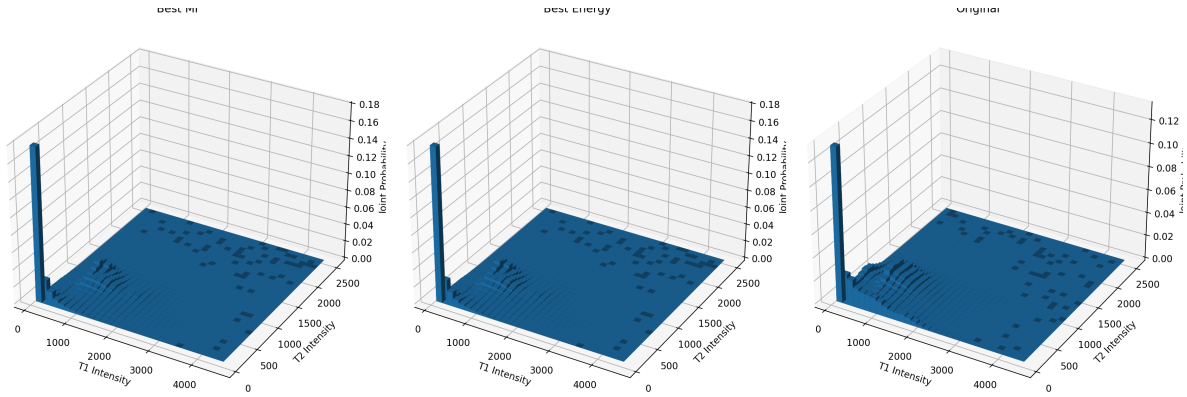


Figure 8: Overview of the joint histogram after registration.

Before the registration, the joint histogram appears scattered, which indicates a poor correlation between T1 and T2. After the registration, the histogram becomes more compact, seeing higher probabilities along the narrow ridge. This shows an increased statistical dependency between the two modalities. The MI and negative-MI methods gives us almost identical histogram distributions, which confirms that both optimization criteria achieves comparable registration performances. In their joint histogram we can observe that their probability is smaller and that they are concentrated around fewer bins. This indicates a better alignment.

8.1 Implementation of Task 6

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 def plot_joint_histogram(T1, T2, title, B=32, threshold=10, ax=None):
5     H, xedges, yedges = np.histogram2d(
6         T1.flatten(),
7         T2.flatten(),
8         bins=B,
9         range=[[threshold, np.max(T1)], [threshold, np.max(T2)]]
10    )
11    jointProb = H / np.sum(H)
12
13    xpos, ypos = np.meshgrid(
14        xedges[:-1] + xedges[1:],
15        yedges[:-1] + yedges[1:]

```

```
16 )
17 xpos = (xpos.flatten() * 0.5)
18 ypos = (ypos.flatten() * 0.5)
19 xpos = xpos.ravel()
20 ypos = ypos.ravel()
21 zpos = np.zeros_like(xpos)
22
23 dx = (xedges[1] - xedges[0]) * np.ones_like(zpos)
24 dy = (yedges[1] - yedges[0]) * np.ones_like(zpos)
25 dz = jointProb.ravel()
26
27 ax.bar3d(xpos, ypos, zpos, dx, dy, dz, shade=True)
28 ax.set_xlabel('T1 Intensity')
29 ax.set_ylabel('T2 Intensity')
30 ax.set_zlabel('Joint Probability')
31 ax.set_title(title)
32
33 # --- Create figure with 3 subplots ---
34 fig = plt.figure(figsize=(18, 6))
35
36 ax1 = fig.add_subplot(131, projection='3d')
37 plot_joint_histogram(T1_data, best_resampled, "Best MI", ax=ax1)
38
39 ax2 = fig.add_subplot(132, projection='3d')
40 plot_joint_histogram(T1_data, best_resampled_ew, "Best Energy", ax=ax2)
41
42 ax3 = fig.add_subplot(133, projection='3d')
43 plot_joint_histogram(T1_data, T2_data_resampled, "Original", ax=ax3)
44
45 plt.tight_layout()
46 plt.show()
```

9 Additional task - Compare the MI obtained with landmark-based and intensity-based registration

Perform landmark-based registration between the T1-weighted and T2-weighted volumes instead, just like you did in the last exercise. For the resulting registration, view your results with the Viewer() class; visualize the joint histogram, and calculate the mutual information.

9 ADDITIONAL TASK - COMPARE THE MI OBTAINED WITH LANDMARK-BASED AND INTENSITY-BASED REGISTRATION

Compare the results with those obtained using mutual information, and discuss.

When looking at figure 9 we can observe that the affine and rigid registrations histograms appear to be more "smeared" than the one from Intensity-based registration. This means that their intensity probabilities are spread out more when comparing to the MI joint histogram.

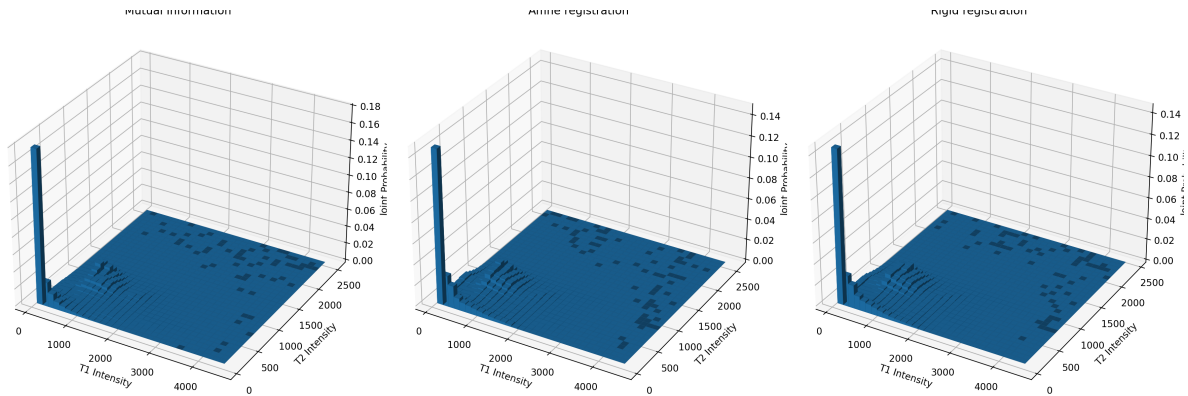


Figure 9: Joint histograms of the affine, rigid, and Intensity-based registration.

Looking at figure 10 we can observe that their registrations look very similar, but the Intensity-based registration appear to be angled a bit in the sagittal and coronal view.

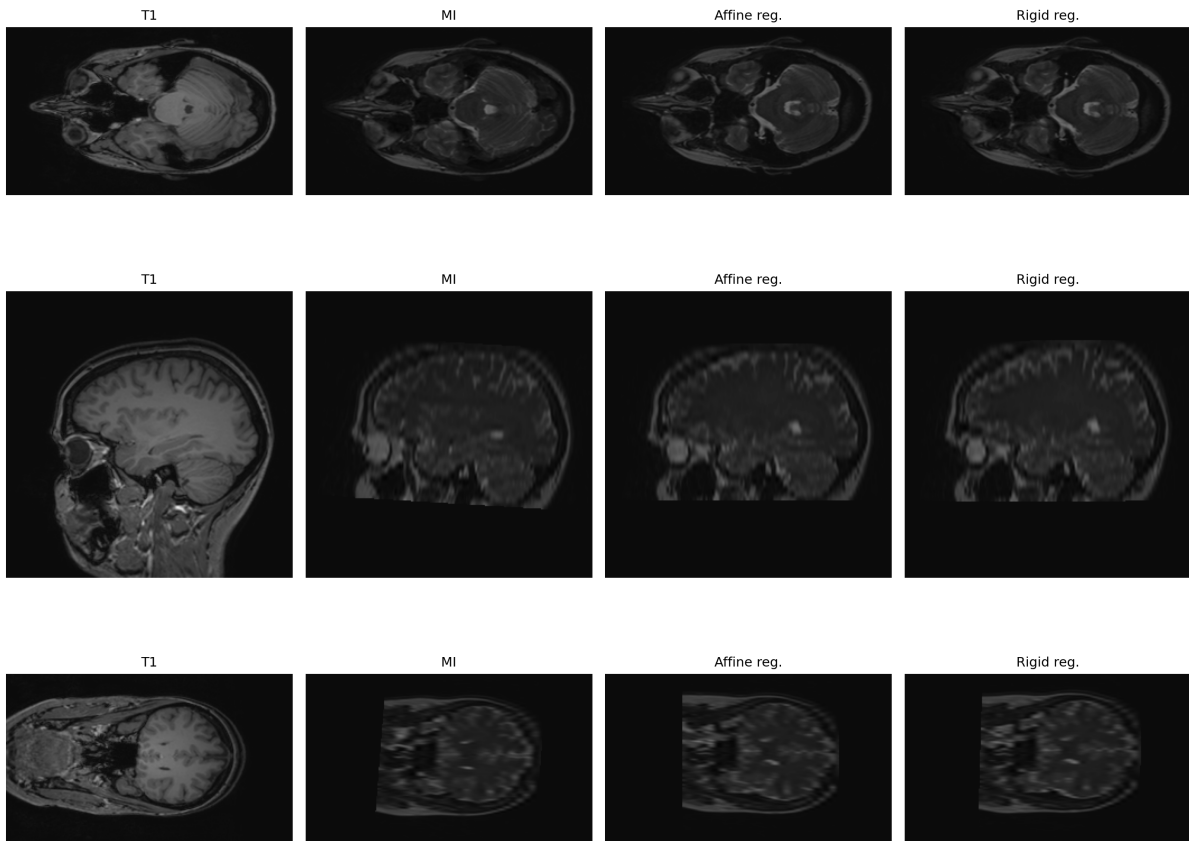


Figure 10: Visualization of the resulting registration using affine, rigid, and Intensity-based registration.

The calculated mutual information for the 3 registration methods were:

- Mutual Information after Affine Registration: -0.1332
- Mutual Information after Rigid Registration: -0.1305
- Mutual Information after Intensity-based registration: -0.0142

The closer to 0 the better the alignment between the two images. From these we can observe that between affine and rigid transformation that rigid transformation has a bit better alignment, when only looking at the mutual information. When comparing both affine and rigid to the Intensity-based registration we can observe a value much closer to 0. This means that the images

share more information and it suggest that the intensity-based registration achieved much better alignment than rigid or affine.

9.1 Implementation of additional task

We started by using the code found last week and finding the affine and rigid transformation.

```
1  #Affine registration
2  #Stacking the chosen landmarks in one matrix for each image
3  LM_t1 = np.vstack([
4      [37, 110, 79, 1],      # Nose ridge
5      [240, 110, 78, 1],     # Back skull
6      [144, 110, 131, 1],    # Right side skull
7      [144, 110, 20, 1],     # Left side skull
8      [144, 198, 75, 1]      # Top of brain/skull
9  ])
10
11 LM_t2 = np.vstack([
12     [125, 231, 8, 1],       # Nose ridge
13     [137, 20, 8, 1],        # Back skull
14     [55, 116, 8, 1],        # Right side skull
15     [206, 116, 8, 1],       # left side skull
16     [128, 116, 25, 1]       # Top of brain/skull
17 ])
18
19 #Converting the landmarks to world coordinates
20 T1_world = (A_T1 @ LM_t1.T).T[:, :3]
21 T2_world = (A_T2 @ LM_t2.T).T[:, :3]
22
23 #Building the X matrix for the equation 2.8
24 X = np.hstack([T1_world, np.ones((T1_world.shape[0], 1))]) # (N, 4)
25
26 #Doing Least squares to find A
27 A = np.linalg.inv(X.T @ X) @ X.T @ T2_world
28
29 # Build full affine matrix M
30 M = np.eye(4)
31 M[:3, :3] = A[:3, :].T    # rotation+scaling
32 M[:3, 3] = A[3, :]        # translation
33 mapping_matrix = np.linalg.inv(A_T2) @ M @ A_T1
34
35 V1,V2,V3 = np.meshgrid( np.arange( T1_data.shape[0] ), np.arange(
    ↪ T1_data.shape[1] ), np.arange( T1_data.shape[2] ), indexing='ij' )
```

9 ADDITIONAL TASK - COMPARE THE MI OBTAINED WITH LANDMARK-BASED AND INTENSITY-BASED REGISTRATION

```
36 V_stack = []
37 V_stack.append( V1.flatten() )
38 V_stack.append( V2.flatten() )
39 V_stack.append( V3.flatten() )
40 V_stack.append( np.ones( V1.size ) )
41 V_stack = np.array(V_stack)
42
43 #Converting the points to T2 space
44 V_T2_space = mapping_matrix @ V_stack
45
46 #Now we interpolate the T2 data to the T1 grid
47 V_T2_in_T1_space_intensity = scipy.ndimage.map_coordinates( T2_data,
48     ↪ V_T2_space[:3,:] )
49 affine_reg = V_T2_in_T1_space_intensity.reshape( T1_data.shape )
50
51 #Rigid registration
52 def rigid_registration(P, Q):
53     centroid_P = np.mean(P, axis=0)
54     centroid_Q = np.mean(Q, axis=0)
55
56     P_centered = P - centroid_P
57     Q_centered = Q - centroid_Q
58
59     H = P_centered.T @ Q_centered
60     U, S, Vt = np.linalg.svd(H)
61     R = Vt.T @ U.T
62
63     if np.linalg.det(R) < 0:
64         Vt[-1, :] *= -1
65         R = Vt.T @ U.T
66
67     t = centroid_Q - R @ centroid_P
68     return R, t
69
70 R, t = rigid_registration(T1_world, T2_world)
71
72 # Step 3: Build transformation matrix
73 M = np.eye(4)
74 M[:3, :3] = R
75 M[:3, 3] = t
76
77 # Step 4: Map T1 voxels to T2 voxels
78 mapping_matrix = np.linalg.inv(A_T2) @ M @ A_T1
79
80 V1, V2, V3 = np.meshgrid(np.arange(T1_data.shape[0]),
```

```
80         np.arange(T1_data.shape[1]),
81         np.arange(T1_data.shape[2]),
82         indexing='ij')
83
84 V_T1 = np.column_stack([V1.flatten(), V2.flatten(), V3.flatten(),
85 ↪ np.ones(len(V3.flatten()))])
86
87 V_stack = V_T1.T
88
89 # Step 5: Interpolation
90 V_T2_in_T1_space_intensity = scipy.ndimage.map_coordinates( T2_data,
91 ↪ V_T2_space[:3,:],order=3 )
92 Rigid_reg = V_T2_in_T1_space_intensity.reshape( T1_data.shape )
```

We could then compare them with the ones found using Intensity-based registration.

```
1 #Comparing the Registrations
2 MI_affine = mutual_information(B, T1_data, affine_reg, threshold)
3 MI_rigid = mutual_information(B, T1_data, Rigid_reg, threshold)
4 MI_MI = mutual_information(B, T1_data, best_resampled, threshold)
5 MI_EW = energy(B, T1_data, best_resampled_ew, threshold)
6 print(f"Mutual Information after Affine Registration: {MI_affine:.4f}")
7 print(f"Mutual Information after Rigid Registration: {MI_rigid:.4f}")
8 print(f"Mutual Information after Rotation: {MI_MI:.4f}")
9 print(f"Mutual Information after Negative MI: {MI_EW:.4f}")
10
11 Mutual Information after Affine Registration: -0.1332
12 Mutual Information after Rigid Registration: -0.1305
13 Mutual Information after Rotation: 0.0142
14 Mutual Information after Negative MI: -0.0142
```

10 Conclusion

In this exercise we implemented intensity based registration using first the mutual information (MI) and then using the negative mutual information, also known as the energy function $E(w)$. We could observe that when comparing to the affine's and rigid's registration mutual information the Intensity-based registration had a better alignment after registration.

Another pro of using the Intensity-based registration was that we did not need to manually find

landmarks in the images, and thereby the registration process was faster, simpler, and more efficient.
